Kotlin Multiplatform - Practice Questions

Sections 1 and 2 with answers | Lecture 8 (What is KMP) and Lecture 9 (Project Structure Fundamentals)

Section 1 - What Is Kotlin Multiplatform (Lecture 8)

Question 1

THEORETICAL

Context: *A teammate says: "KMP is basically like Flutter or React Native, just for Kotlin developers."*

Task: *Explain why KMP is neither a bridge nor a VM; name what Flutter renders through, what React Native talks to native views across, and what Kotlin/Native produces for iOS.*

Answer

KMP is a **compiler technology**: you write logic once in Kotlin and it is compiled natively for every target. There is no embedded runtime translating calls - shared code becomes native code on each platform. The lecture's contrast:

Technology	How it works
Flutter	Renders its **own widget tree through Skia / Impeller** - it does not use native platform views.
React Native	Talks to native views **across a JS bridge** (or the newer JSI).
Kotlin Multiplatform	Skips the middle layer. On iOS, **Kotlin/Native uses LLVM to produce real ARM64 binaries** (not an interpreter); on Android it targets the JVM. The UI can stay fully native, or be shared with Compose Multiplatform.

Friendly framing: "The goal is similar - less duplicated work - but the mechanism is different. KMP has no rendering engine or bridge in the middle; it only shares the logic, and the result is a real native binary."

Question 2

THEORETICAL

Context: *Android and iOS teams write the same validation, networking, and business-rule code twice, once in Kotlin and once in Swift.*

Task: *Describe the "two codebases, one product" problem as the lecture frames it, and state the three consequences of duplication.*

Answer

The problem: the Android team writes validation, networking, and business rules in Kotlin; the iOS team writes the exact same validation, networking, and business rules again in Swift. Two languages, two codebases, one product.

Three consequences:

- **Duplicated bugs** - the same defect has to be found and fixed twice.
- **Inconsistent behavior between platforms** - the two implementations drift apart.
- **Double the QA effort** for every business-logic change.

KMP's fix: write that logic once in `commonMain`.

Question 3

THEORETICAL

Context: *A product manager asks what percentage of the app can be shared and which features are realistic to share vs. keep native.*

Task: *Give the typical shared percentage range, and two examples each of shared logic and logic that stays native, with the reason for each native example.*

Answer

Typical range: most production teams share **60-80%** of their codebase - "the logic, not always the pixels."

Typically shared (commonMain)	Usually kept native (with reason)
Networking & API clients - e.g. Ktor as a shared HTTP client	**Platform UI** - unless you also adopt Compose Multiplatform
Validation & business rules - logic that must behave identically on both platforms	**Camera, biometrics, sensors** - deep platform-hardware integration

Other items on the slide: local storage and caching (SQLDelight, Room KMP) and state management (shared ViewModels) are shared; push notifications (platform-specific delivery services) and some navigation (platform-idiomatic transitions and deep links) stay native.

Question 4

THEORETICAL

Context: *The team wants to know which KMP targets are safe to ship to production today and which are still risky.*

Task: *Name every target listed as Stable and the one listed as Beta, and state what Beta implies for a production feature.*

Answer

Target	Stability
Android	Stable
iOS (Arm64 + Simulator)	Stable
Desktop - Windows, macOS, Linux	Stable
Server / backend (e.g. with Ktor)	Stable
Web - Kotlin/JS	Stable
Web - Kotlin/Wasm	**Beta**

What Beta implies: the target is not yet at the full stability level JetBrains guarantees, so APIs, tooling and behavior may still change and rough edges are expected. A team can use it, but should treat it as a risk for a production feature - accept possible migration work, keep it isolated, or wait. (The slide only labels it Beta; this implication is the standard meaning of the label.)

Question 5

HANDS-ON - ANDROID STUDIO

Context: *The lecture's expect/actual example: common declaration in commonMain/Platform.kt, implementations in androidMain/Platform.android.kt and iosMain/Platform.ios.kt.*

Task: *Create a KMP project targeting Android and iOS; declare expect fun getPlatform(): Platform with a Platform type having name: String; add actual implementations; build; screenshot the three files.*

Answer

Steps: File > New > New Project > *Kotlin Multiplatform* (Android + iOS), then edit the three files below. Run *Build* > *Make Project*; both `actual` declarations should resolve with no red underlines. Take your screenshot with the three files open side by side.

`commonMain/Platform.kt`

```
interface Platform {
    val name: String
}

expect fun getPlatform(): Platform
```

`androidMain/Platform.android.kt`

```
class AndroidPlatform : Platform {
    override val name: String = "Android"
}

actual fun getPlatform(): Platform = AndroidPlatform()
```

`iosMain/Platform.ios.kt`

```
class IOSPlatform : Platform {
    override val name: String = "iOS"
}

actual fun getPlatform(): Platform = IOSPlatform()
```

Why it works: the compiler merges each `expect` declaration with its `actual` into one API per platform. Both files must use the same package and signature as the expect. Note: expect/actual *functions* are fine; expect/actual *classes* are still Beta, which is why the example uses an interface plus a function.

Question 6

THEORETICAL

Context: *A colleague wants to share the UI layer too and is unsure whether that needs a different tool than KMP.*

Task: *Explain the relationship between KMP and Compose Multiplatform: what each shares, stability by platform, and why KMP can be used without CMP but never the reverse.*

Answer

	Kotlin Multiplatform	Compose Multiplatform
Shares	Logic: networking, data, business rules, state. UI can stay 100% native (Jetpack Compose on Android, SwiftUI on iOS).	JetBrains' declarative UI framework that **additionally shares the UI layer**.
Stability	Stable since Nov. 2023	**Stable** on Android, iOS and desktop; **Beta** on web (Wasm-based target)

Why one-directional: Compose Multiplatform is built *on top of* the multiplatform foundation (KMP) - it needs KMP's targets, source sets and compiler support to exist. KMP, however, is complete by itself for sharing logic. So: *"You can use Kotlin Multiplatform without Compose Multiplatform - but not the other way around."* They are not competing tools; they are two layers, often used together.

Section 2 - KMP Project Structure Fundamentals (Lecture 9)

Question 7

THEORETICAL

Context: *A student writes* `import java.io.File` *inside a function in commonMain; the IDE flags it as unresolved, though it compiles in a plain JVM project.*

Task: *Explain what "common code" means, and why code that only exists on some targets cannot be referenced from commonMain.*

Answer

Common code is code shared across platforms and placed in `commonMain`: *one source, many binaries*. The compiler takes that one common source as input and produces a set of platform-specific binaries from it - not copies, the same file compiled differently for each target.

Why java.io.File fails: `java.io.File` is part of the JDK. commonMain does not compile only to the JVM - it also compiles to native (iOS) and JS targets, where the JDK simply does not exist. Since the same file must compile for *every* declared target, Kotlin refuses to compile a reference that would be broken on any one of them. A plain JVM project has only one target (the JVM), so there the import is always valid.

Fix: move the code to `jvmMain` (platform source sets can use platform-only APIs), or hide it behind expect/actual. The same applies to JVM-only extras of multiplatform libraries, e.g. `CoroutineDispatcher.asExecutor()` in kotlinx.coroutines.

Question 8

HANDS-ON - ANDROID STUDIO

Context: *Targets are "labels" attached to source files through the Gradle DSL; minimal example* `kotlin { jvm(); iosArm64() }`.

Task: *Declare jvm() and iosArm64() in build.gradle.kts, sync, and record the exact source set folder names Gradle creates.*

Answer

`build.gradle.kts`

```
plugins {
    kotlin("multiplatform") version "<your Kotlin version>"
}

kotlin {
    jvm()         // JVM target
    iosArm64()    // 64-bit iPhones
}
```

Expected result after Gradle sync (Project view, `src/`): one shared source set plus one per declared target, each with a Main and a Test counterpart:

Source set	Role
`commonMain` / `commonTest`	Shared by every declared target
`jvmMain` / `jvmTest`	JVM target only
`iosArm64Main` / `iosArm64Test`	iosArm64 target only

Note: the IDE shows a source set folder only when it exists on disk. If a folder is missing, create it (or use *Project Structure* / the Gradle tool window to view source sets). Depending on your Kotlin version, the default hierarchy template may also register intermediate sets such as `appleMain`, `iosMain` and `nativeMain`; record whatever you actually see in your own project.

Question 9

THEORETICAL

Context: *Two developers disagree about what makes something a "source set" versus just a folder of Kotlin files.*

Task: *List the four defining properties of a source set, and explain what distinguishes commonMain (root) from androidMain (platform/leaf).*

Answer

A source set is a set of files with its own targets, dependencies, and compiler options. The four properties:

1. Has a **name that is unique** within the project.
2. Contains **source files**, usually in a directory of the same name.
3. Specifies **which targets** its code compiles to.
4. Defines its **own dependencies and compiler options**.

A plain folder has none of this metadata - it is just files. Source sets are what Kotlin knows about via the Gradle DSL (e.g. `sourceSets { commonMain { /*...*/ } }`).

Root vs. leaf: `commonMain` is the one predefined source set in every KMP project, sits at the root of the hierarchy, and compiles to *all* declared targets - so it may only use APIs available everywhere. `androidMain` is a platform (leaf) source set: it compiles to *one* target (Android), so it can use platform-only APIs, and it can see commonMain, but commonMain can never see back into it.

Question 10

THEORETICAL

Context: *Compiling for iosArm64 merges code from three source sets into one program; a junior asks how Kotlin decides which to include.*

Task: *State the compilation merge rule, and list the four golden rules, including the direction information may flow between commonMain and a platform source set.*

Answer

Merge rule: Kotlin **gathers every source set labeled for that target, then compiles them together** as one unit. For iosArm64 that chain is `commonMain + appleMain + iosArm64Main` - merged into one native executable. Source sets carrying a different label (e.g. `jsMain`, `jvmMain`) are simply left out of that compilation.

Four golden rules:

1. **Declare a target** before Kotlin will compile anything to it.
2. Code shared by everything belongs in **commonMain**.
3. Code for one target belongs in **that target's own source set**.
4. Platform source sets can see commonMain, but **commonMain can never see back into them**.

Direction of information: one way only - from general (commonMain) down to specific (platform source sets), never the reverse.

Question 11

HANDS-ON - ANDROID STUDIO

Context: *Practice expect/actual on a different example than the platform accessor in Lecture 8.*

Task: *Add expect fun currentTimestamp(): Long to commonMain; implement actuals in androidMain and iosMain; build for both targets.*

Answer

`commonMain/Timestamp.kt`

```
expect fun currentTimestamp(): Long
```

`androidMain/Timestamp.android.kt`

```
actual fun currentTimestamp(): Long = System.currentTimeMillis()
```

`iosMain/Timestamp.ios.kt`

```
import platform.Foundation.NSDate
```

```
actual fun currentTimestamp(): Long =
    (NSDate().timeIntervalSince1970 * 1000).toLong()
```

Important detail: `timeIntervalSince1970` returns *seconds* as a Double, while `System.currentTimeMillis()` returns *milliseconds*. Multiplying by 1000 makes both platforms return the same unit, which is the whole point of a shared contract. Keep the same package and file-level signature in all three files; build with *Build > Make Project* and check that no `actual` is reported as missing.

Question 12

HANDS-ON - ANDROID STUDIO

Context: *Adding a third target, js(), to jvm(); iosArm64() shows that Kotlin creates a matching source set automatically.*

Task: *Add the JS target, re-sync, and verify new jsMain/jsTest source sets appear with no manual wiring.*

Answer

```
kotlin {
    jvm()
    iosArm64()
    js(IR) {          // on newer Kotlin versions: js { browser() }
        browser()
    }
}
```

Expected result: after sync, `jsMain` and `jsTest` appear next to `commonMain/Test`, `jvmMain/Test` and `iosArm64Main/Test`. (If the folders are not visible, create them in `src/`.)

Why no wiring is needed: declaring a target attaches a third label to every common file. The source code did not change - no new file, no conditional logic - Kotlin just routes the same common code through a third compiler backend (Kotlin/JS) and produces a `.js` output alongside the JVM class files and the iOS native executable. Syntax note: `js(IR)` is deprecated in recent Kotlin releases because IR is the default; use `js { browser() }` if your IDE warns.

Question 13

HANDS-ON - ANDROID STUDIO

Context: *The lecture's testing example pairs Greeting in commonMain with GreetingTest in commonTest.*

Task: *Create Greeting.greet() in commonMain and a GreetingTest in commonTest using @Test and assertEquals; run it and confirm it passes.*

Answer

`commonMain/Greeting.kt`

```
class Greeting {
    fun greet(): String = "Hi"
}
```

`commonTest/GreetingTest.kt`

```
import kotlin.test.Test
import kotlin.test.assertEquals

class GreetingTest {
    @Test
    fun testGreet() = assertEquals("Hi", Greeting().greet())
}
```

Run it: click the green gutter icon next to the test class and pick a target, or run `./gradlew allTests` (e.g. `jvmTest`, `iosSimulatorArm64Test`). commonTest needs the `kotlin("test")` dependency (included by the project wizard). **Why it follows the lecture:** every Main source set has a matching Test counterpart wired automatically, and a test in commonTest compiles to every declared target - *write one test, run it everywhere*.